

Making Everstrife: Using a 3D Game Build in Unity

Zhire Pelayo
Christopher Kulak
Team 3

MacEwan University
CMPT 330 - AS01 - Introduction to Real-Time Gaming
Dr. Sam Qorbani
Apr 1, 2026

Table of Contents

Abstract

1 Introduction

1.1 Overview

1.2 Initial Goals

2 Game Description

2.1 Game Summary

2.2 Player Character

2.3 Enemies

2.4 Environment

3 Engine Choice

4 Development

4.1 Planning and Outline

4.2 Constructing Game Level

4.3 General Game Features

5 Technical Challenges

6 Game assets

6.1 Texture and Asset

6.2 SFX

7 Conclusion

8 Future Work

9 Appendix

Abstract

The following paper is a walkthrough of the development, planning and procedures used to make the game of Everstrife by Christopher Kulak and Zhire Pelayo. Everstrife is a top down 2D adventure game set in a medieval fantasy world. Drawing heavily on titles like Diablo 2 and other action-adventure RPG games, Everstrife seems to capture that classic feel of dungeon crawling, exploration and player customization the genre is known for. This paper outlines the steps and procedures used to make such a game from its inception to its development into a vertical slice.

1 Introduction

1.1 Overview

Everstrife is a top-down action-adventure role-playing game (RPG) developed using the Unity game engine. The project was created as part of CMPT 330 with the goal of exploring real-time game development concepts, including player input systems, artificial intelligence, physics, and user interface design. Although the game presents itself visually as a 2D experience, it is built using Unity's 3D engine to take advantage of enhanced rendering, lighting, and spatial systems.

The development process followed an iterative approach, beginning with initial concept design and progressing toward a functional vertical slice. This allowed for early testing of core gameplay mechanics such as movement, combat, and enemy interactions before expanding into more complex systems. The project emphasizes both technical implementation and game design principles, demonstrating how core systems interact to create a cohesive gameplay experience.

1.2 Initial Goals

The initial goal of Everstrife was to develop a playable vertical slice that captures the core gameplay loop of a classic action RPG. This included implementing responsive player movement, combat mechanics (both melee and ranged), basic enemy AI, and an interactive environment.

Additional goals included:

- Creating an intuitive user interface for inventory and player menus
- Designing a small but immersive environment that demonstrates open world exploration
- Implementing a basic AI system using A* pathfinding techniques
- Experimenting with layered rendering to simulate depth in a top-down perspective
- Establishing a foundation for future features such as factions, multiple classes, and expanded world-building

Due to time constraints, the focus remained on delivering a polished core experience rather than a fully complete game.

2 Game Description

2.1 Game Summary

The player assumes the role of a Ranger, a versatile class designed to balance close-range and long-range combat. This dual-weapon system allows players to dynamically adapt to different combat situations and give the player a sense of fast based heart racing combat.

- **Melee Combat:** The ranger uses a sword for close-range encounters, allowing for quick attacks when enemies are nearby.
- **Ranged Combat:** The bow enables attacking enemies from a distance, encouraging strategic positioning and player movement
- **Mobility:** The dash mechanic provides temporary speed boosts, allowing players to evade attacks or reposition during combat. There is also no cooldown limit on the dash encouraging players to repeatedly dash making player controls more skill based.

The player system is built around Unity's input system, ensuring responsive and customizable controls. The design prioritizes fluid movement and combat transitions to maintain engagement.

2.2 Player Character



- You play as a ranger. Ranger has two weapon types: a sword for close-range combat, and bows and arrows for ranged attacks.

2.3 Enemies

Enemies in Everstrife vary in behavior and difficulty, providing different combat challenges:

- **Slime:** Basic enemy with simple movement and low physical damage
- **Skeleton:** Slightly more advanced enemy with faster movement and ranged damage
- **Zombie:** Slower but more durable enemy, encouraging sustained combat
- **Dragon:** A high-level enemy (or boss-type) designed to introduce more complex mechanics

All enemies use pathfinding to navigate the environment and pursue the player. Their behaviors are controlled through scripts that handle movement, attacks, damage, and loot drops.



- Slime



- Skeleton



- Zombie



- Dragon

2.4 Environment

The initial playable area is set in a simple plains biome, establishing the beginner or tutorial area the player would expect from a fantasy game. The grass is green, the monuments aren't too dark or overtaken by plants. This environment includes:

- Tombstones, trees, benches and ruins to create atmosphere
- Pathways and obstacles that guide player movement
- Layered terrain to simulate vertical depth, mainly with trees and leaves
- Interactive elements such as doors and pressure plate puzzles

The environment design focuses on balancing visual appeal with gameplay functionality, ensuring that obstacles and terrain contribute meaningfully to player navigation and combat.



- The first area takes place in a graveyard.

3 Engine Choice

For Everstrife we chose the Unity Engine version 2022.3.62f3. Using the pre built 3D (Built in render pipeline). One primary advantage is full three-dimensional spatial control. In the 2D workflow, physics and movement systems like Rigidbody2D and Collider2D operate strictly on the X and Y axes, with Z-depth used only for visual layering. In contrast, the 3D workflow allows objects to exist and interact across all three axes, enabling more nuanced control over depth, collision, and positioning.

The 3D pipeline also enables native support for meshes and true rotation. While 2D workflows rely on sprites, the 3D system allows objects to be rotated dynamically without requiring separate pre-rendered images for each angle. This can significantly reduce asset production overhead and enable more dynamic visual behavior. Another important factor is camera flexibility. This opens the door for more impressive and immersive lighting details we could use for our game.

4 Implementation

4.1 Planning and Outline

Development followed an agile-like workflow, where progress was divided into weekly milestones. Each week included planning, implementation, and review phases. This allowed for continuous iteration and improvement of game features.

Tasks were divided based on feature ownership, such as:

- Player systems (movement, combat)
- Health, stamina and mana bars

- Enemy AI and behavior
- UI and inventory systems
- Level design and environment

We prioritized building a vertical slice early, ensuring that all major systems were functional before expanding content. This allowed us to create a lot of features and content early and build towards that demo of a vertical slice.

4.2 Constructing Game Level

We found an 2D asset pack “Pixel Art TopDown” that matched our theme and feel we were going for with Everstrife. We wanted to have a lot of open grasslands, bogs, swamps, dark cliffs, large trees and abandoned villages to cover our game world. We created our vertical slice tutorial area with grass, rocks, flowers and trees to give the world depth. Since we are a top down 2D game in order to simulate changing depth (either going up or down) we use wall tiles, stairways and game shadows to give the feel of a 3D game. We broke our game down into UI layers to make impassable objects, objects the player can pass behind and boundaries for the game world. Default was layer 0, Ground was layer 1 used for the game ground tiles, layer 2 was the stairs layer which lays over the ground layer, layer 3 which is walk in front, layer 4 which is collision layer, used for walls, impassable rocks and tree trunks. Layer 5 is the player layer, layer 6 is the walk behind layer mainly used for the leaves of trees or tunnels the player may walk underneath and lose visibility of the player character temporarily.

4.3 General Game Features

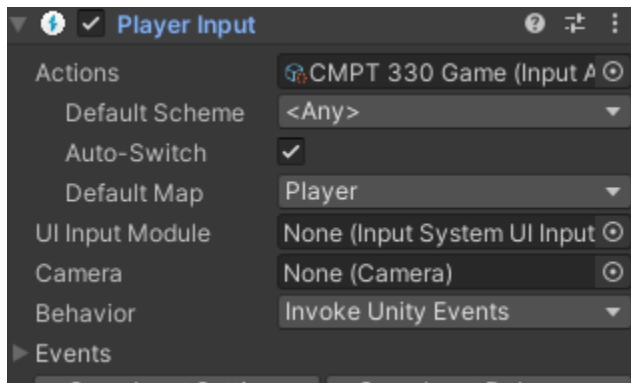
Everstrife is an action adventure RPG so it has a lot of the features standard to this genre.

Controls

Controls	Inputs
Movement	WASD or arrow keys. W (or up arrow) for up, S (or down arrow) for down, A for left (or left arrow), D for right (or right arrow).
Dashing	Space Key
Ranged Attack/Selecting	Left-click will attack when the mouse isn't hovering over an inventory
Melee Attack	Right click to draw out your sword
An interact key	E key to open chests
Inventory	I key
Pause Menu	Escape key to pause

Player

- Player Input is managed by the built-in Unity input system



- Player Dash movement is managed by the Player Movement script. The same script also manages the trail effect when dashing and the associated SFX.

```
void Update()
{
    rb.velocity = moveInput * moveSpeed;

    // player dash, direction depending on what button is pressed
    if (canDash && Input.GetKeyDown(KeyCode.Space))
    {
        playSFX();

        if (Input.GetKey(KeyCode.W) && Input.GetKey(KeyCode.D))
        {
            StartCoroutine(Dash(new Vector2(1f, 1f)));
        }

        else if (Input.GetKey(KeyCode.S) && Input.GetKey(KeyCode.D))
        {
            StartCoroutine(Dash(new Vector2(1f, -1f)));
        }

        else if (Input.GetKey(KeyCode.A) && Input.GetKey(KeyCode.W))
        {
            StartCoroutine(Dash(new Vector2(-1f, 1f)));
        }

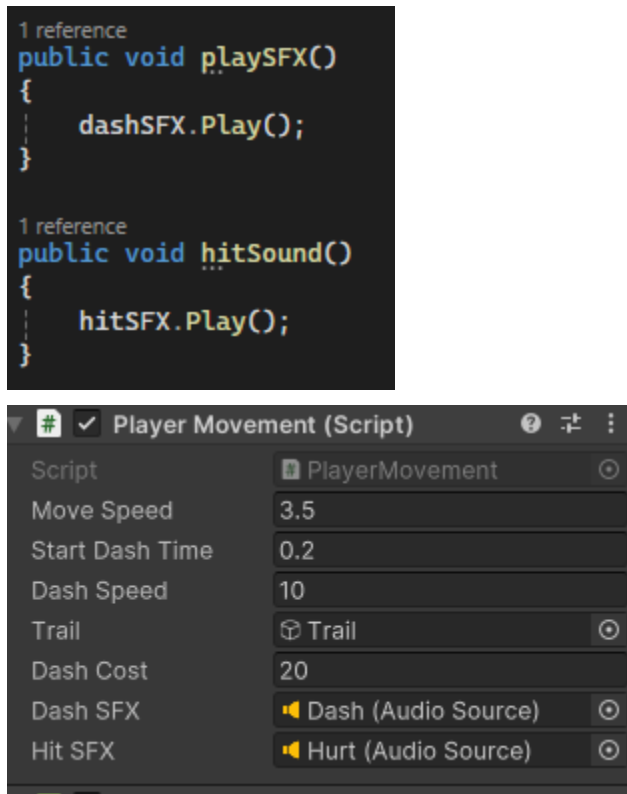
        else if (Input.GetKey(KeyCode.A) && Input.GetKey(KeyCode.S))
        {
            StartCoroutine(Dash(new Vector2(-1f, -1f)));
        }

        else if (Input.GetKey(KeyCode.W))
        {
            StartCoroutine(Dash(Vector2.up));
        }

        else if (Input.GetKey(KeyCode.A))
        {
            StartCoroutine(Dash(Vector2.left));
        }

        else if (Input.GetKey(KeyCode.S))
        {
            StartCoroutine(Dash(Vector2.down));
        }

        else if (Input.GetKey(KeyCode.D))
        {
            StartCoroutine(Dash(Vector2.right));
        }
    }
}
```

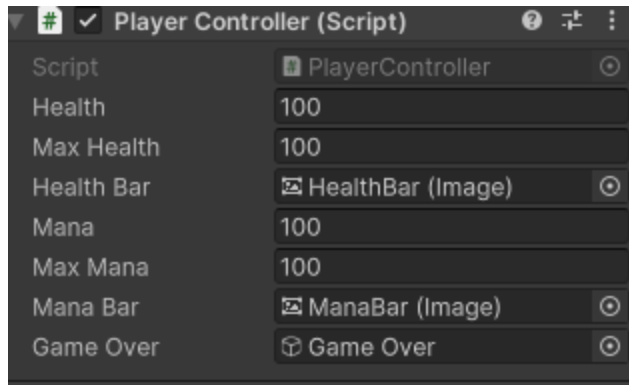


Health and Mana system

- The UI associated with the health and mana system is implemented using canvas and panel on the player screen



- The player's health is managed by the script PlayerController. Due to time constraints, we were unable to implement a proper mana system, but the skeleton logic is included in the script.



```

// Unity Script (2 asset references) 1/2 references
public class PlayerController : MonoBehaviour
{
    public float health;
    public float maxHealth;
    public Image healthBar;

    public float mana;
    public float maxMana;
    public Image manaBar;

    public GameObject gameOver;

    // Start is called before the first frame update
    // Unity Message | 0 references
    void Start()
    {
        Time.timeScale = 1;
        maxHealth = health;
        maxMana = mana;
    }

    // Update is called once per frame
    // Unity Message | 0 references
    void Update()
    {
        healthBar.fillAmount = Mathf.Clamp(health / maxHealth, 0, 1);
        manaBar.fillAmount = Mathf.Clamp(mana / maxMana, 0, 1);

        if (health <= 0)
        {
            Time.timeScale = 0;
            gameOver.SetActive(true);
        }
    }
}

```

- The player's health is reduced in the EnemyScript and EnemyProjectile if an enemy or enemy projectile collides with the Player.

```

// Unity Message | 0 references
private void OnCollisionEnter2D(Collision2D other)
{
    // checks if collides with the player, if so, reduce player hp
    if (other.gameObject.CompareTag("Player"))
    {
        playSFX();
        other.gameObject.GetComponent<PlayerController>().health -= damage;
    }
}

```

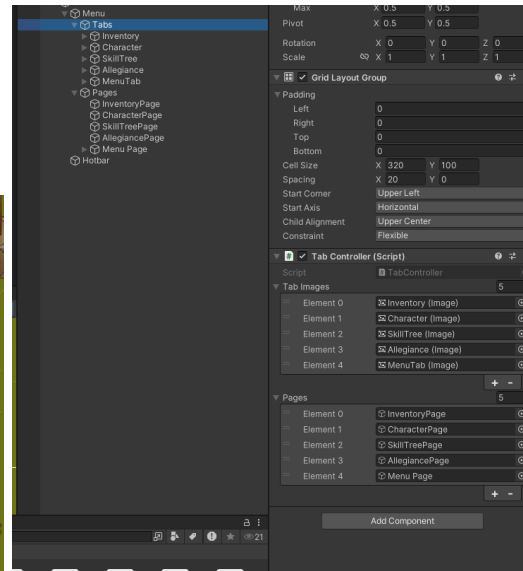
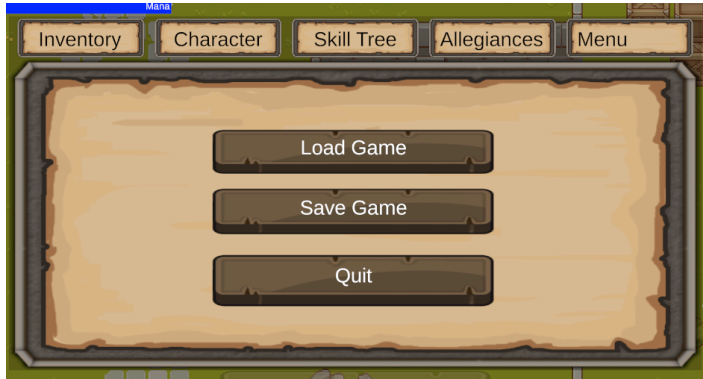
```

private void OnTriggerEnter2D(Collider2D other)
{
    // checks if it collides with player, if so, reduce health
    if (other.CompareTag("Player"))
    {
        Destroy(gameObject);
        other.gameObject.GetComponent<PlayerController>().health -= 15f;
        Debug.Log("player hit");
    }
}

```

Menu UI, Tab switching, and Inventory system

- For the inventory system we used a UI canvas and then made individual images of tabs and pages, where the tab. For the tabs, we have a tabController script that handles which tab is active and greys out which one isn't. For the inventory itself I made a slot prefab object and made a script to duplicate the number of slots based on a variable input.



```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class InventoryController : MonoBehaviour
6  {
7      private IDictionary itemDictionary;
8
9      public GameObject inventoryPanel;
10     public GameObject slotPrefab;
11     public int slotCount; //for future use, in case you want to upgrade bag slots
12     public GameObject[] itemPrefabs;
13     // Start is called before the first frame update
14     @ UnityEngine
15     void Start()
16     {
17         itemDictionary = FindObjectsOfType<ItemDictionary>();
18     }
19
20     public bool AddItem(GameObject itemPrefab)
21     {
22         //Look for an empty slot in our inventory
23         foreach (Transform slotTransform in inventoryPanel.transform)
24         {
25             Slot slot = slotTransform.GetComponent<Slot>(); //get the slot component of the current slot transform
26             if (slot != null && slot.currentItem == null)
27             {
28                 GameObject newItem = Instantiate(itemPrefab, slotTransform); //create a new item of this item prefab
29                 newItem.GetComponent<RectTransform>().anchoredPosition = Vector2.zero; //center the item in the slot
30                 slot.currentItem = newItem; //set the current item of the slot to this new item
31                 return true; //item added successfully
32             }
33             Debug.Log("Inventory is full!");
34         }
35         return false;
36     }
37
38     public List<InventorySaveData> GetInventoryItems()
39     {
40         List<InventorySaveData> invData = new List<InventorySaveData>();
41         foreach (Transform slotTransform in inventoryPanel.transform)
42         {
43             List<InventorySaveData> invData = new List<InventorySaveData>();
44             foreach (Transform slotTransform in inventoryPanel.transform)
45             {
46                 Slot slot = slotTransform.GetComponent<Slot>();
47                 if (slot.currentItem != null)
48                 {
49                     Item item = slot.currentItem.GetComponent<Item>();
50                     invData.Add(new InventorySaveData { ItemID = item.ID, slotIndex = slotTransform.GetInstanceID() }); //get sibling index gets the index of :
51                 }
52             }
53             return invData;
54         }
55     }
56
57     public void SetInventoryItems(List<InventorySaveData> inventorySaveData)
58     {
59         foreach (Transform child in inventoryPanel.transform)
60         {
61             Destroy(child.gameObject);
62         }
63         //create new slots
64         for (int i = 0; i < slotCount; i++)
65         {
66             Instantiate(slotPrefab, inventoryPanel.transform); //sets inventory panel to parent of slot prefabs
67         }
68         foreach (InventorySaveData data in inventorySaveData)
69         {
70             if (data.slotIndex < slotCount)
71             {
72                 Slot slot = inventoryPanel.transform.GetChild((data.slotIndex).GetComponent<Slot>());
73                 GameObject itemPrefab = itemDictionary.GetComponent<Item>(data.ItemID);
74                 if (itemPrefab != null)
75                 {
76                     GameObject item = Instantiate(itemPrefab, slot.transform); //create a new item of this item prefab from the item dictionary
77                     item.GetComponent<RectTransform>().anchoredPosition = Vector2.zero;
78                     slot.currentItem = item;
79                 }
80             }
81         }
82     }
83 }

```

- For save data of player location and the inventory state, we just used a simple json file to store the information

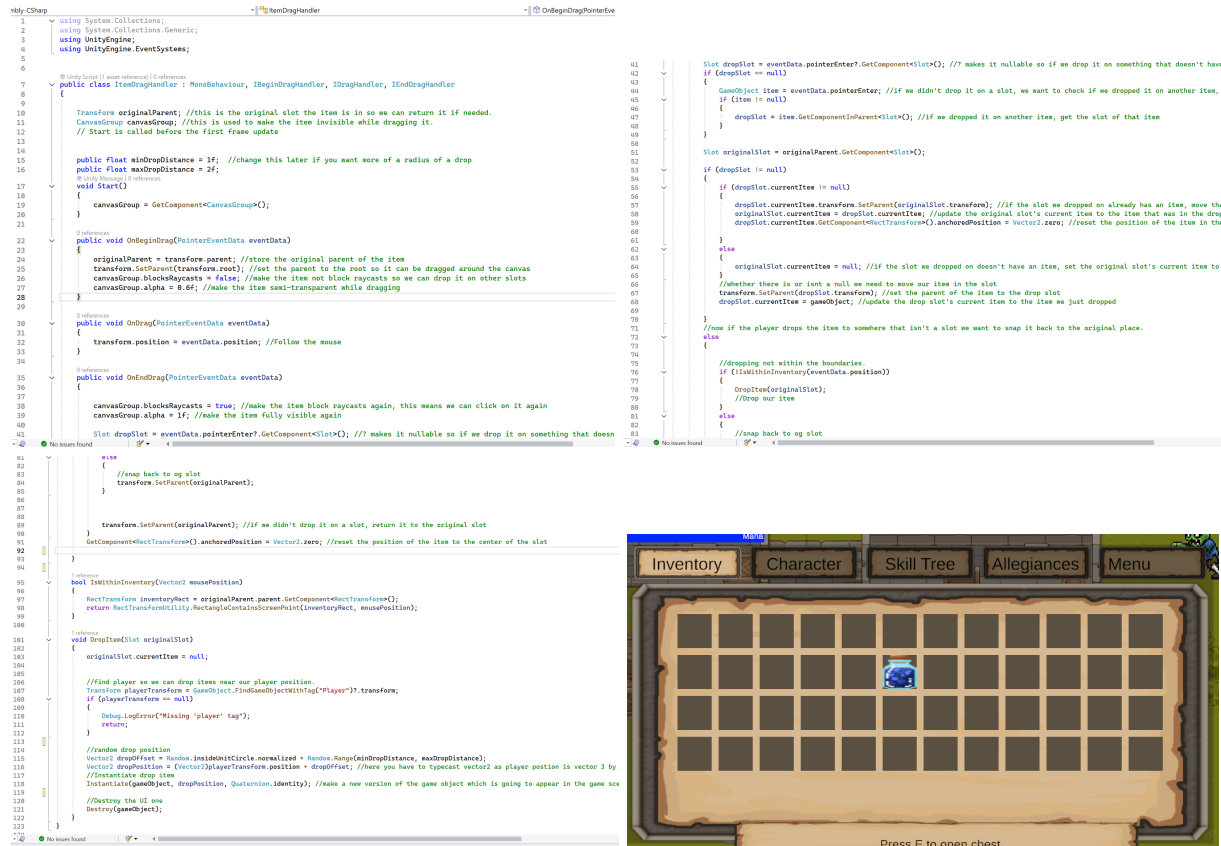
```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  [System.Serializable]
6
7  //this is needed to save data ^^^
8  public class InventorySaveData
9  {
10     public int itemID;
11     public int slotIndex; //this is the index of the slot in the in
12 }
13
14

```

- For moving objects through the inventory as well as for dropping them on the ground we have an itemDragHandler script which checks if the items are oriented into a slot if the item is dragged to an illegal location it snaps back to its initial position, if the object is dragged outside

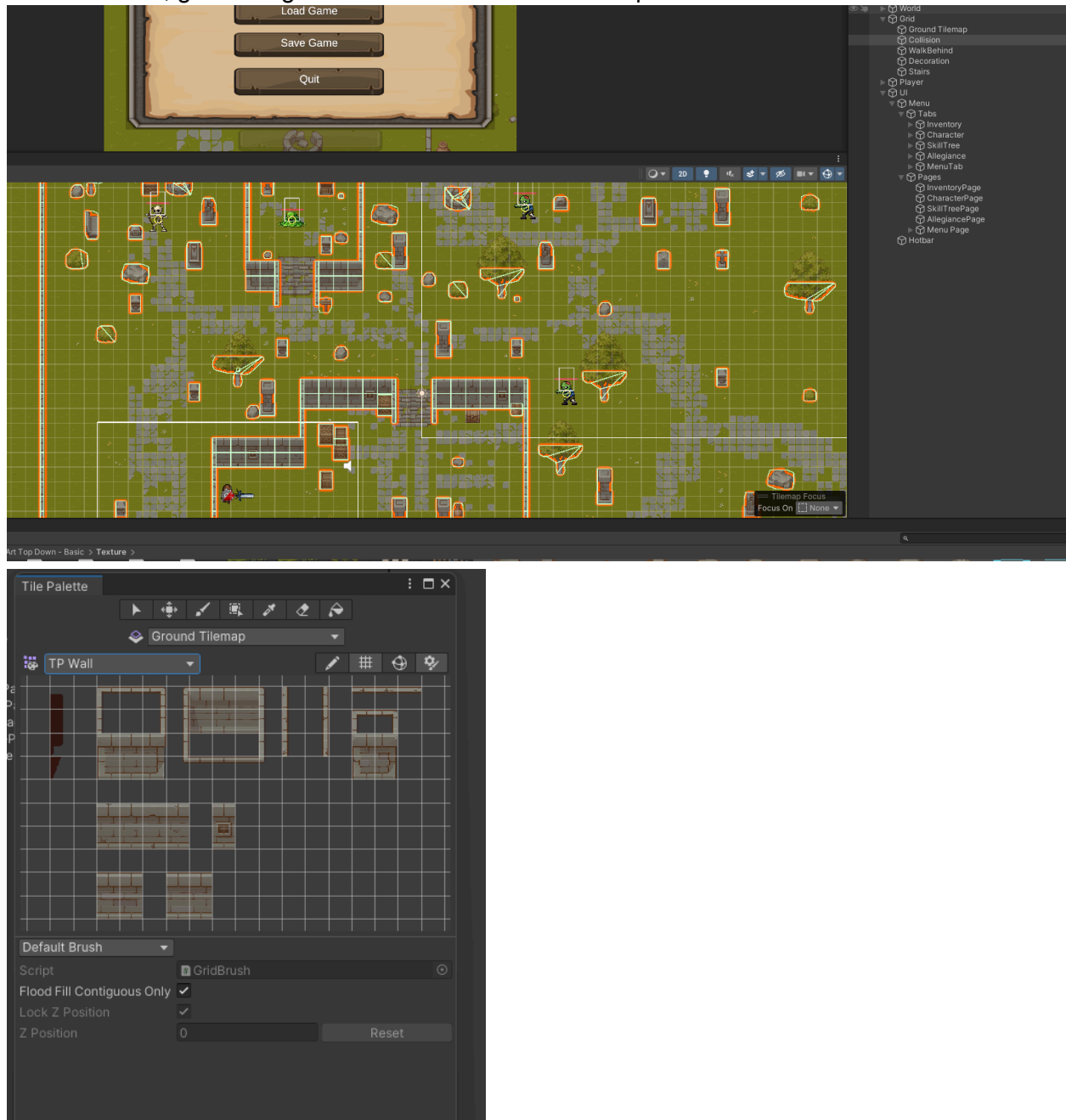
of the game menu, the item is destroyed and another copy of the item is instantiated into the real world as a world drop. This allows us to move items into and out of the inventory into the game world if need be.



Textures and Gameworld

- Using a tile palette and assets we were able to create our game background and using layers give various fields of depth to our game world. Various layers such as a collision layer for obstacles, walk behind for foliage or the backs of pillars and ground tilemaps that the player can

walk in front of, gave our game a sense of realism and depth to the environment.

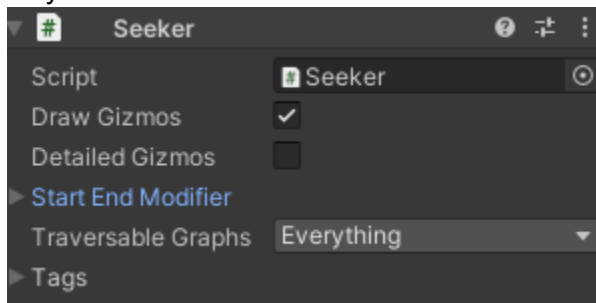


A* Pathfinding Project

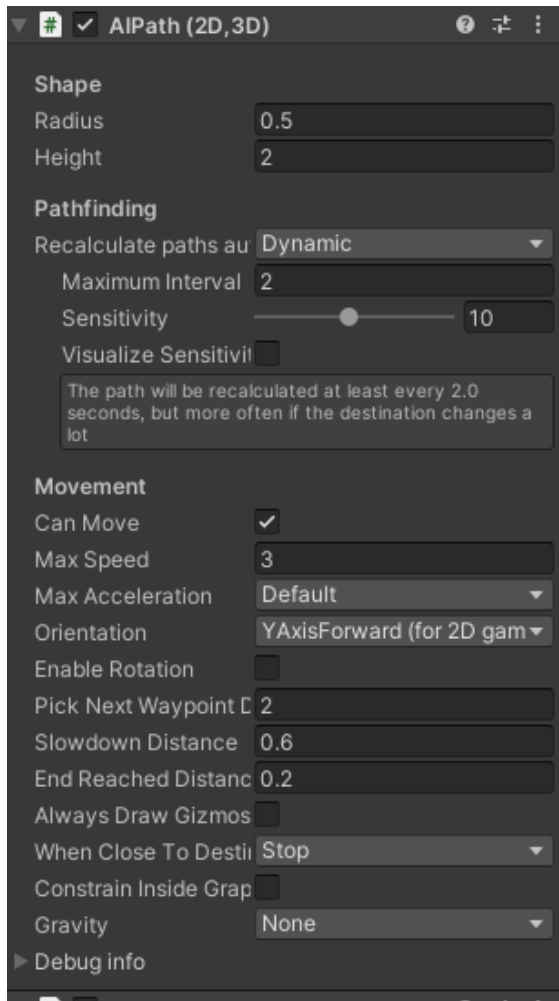
- AI in Everstrife uses A* to pathfind towards the player. To implement, we imported the project from the official website to Unity. Then, you set up the grid system by adding layers that are inaccessible, in our case, the Wall layer.



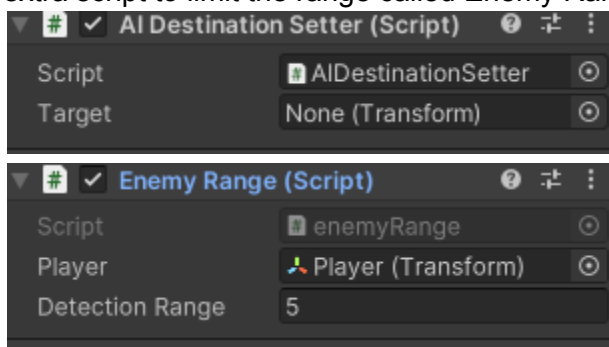
- Pathfinding works by adding three components to an Enemy game object: Seeker, AIPath (2D, 3D), and AI Destination Setter.
- 1. Seeker - this component calculates the shortest route from the Enemy location to the Player.



- 2. AIPath (2D, 3D) - this controls the speed, acceleration, and rotation of the Enemy object.

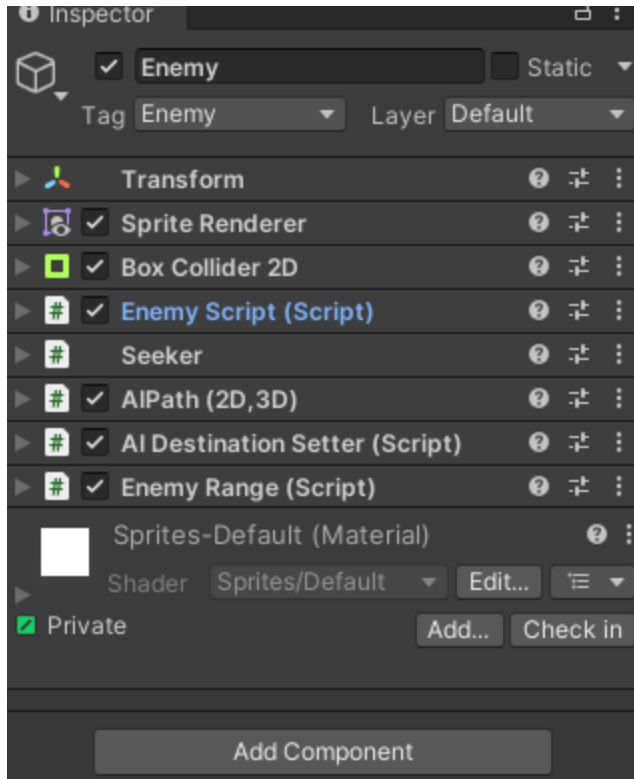


3. AI Destination Setter - sets the destination of the Enemy. Although this would make it so the enemy keeps track of the player's location, no matter the distance. So, I added an extra script to limit the range called Enemy Range



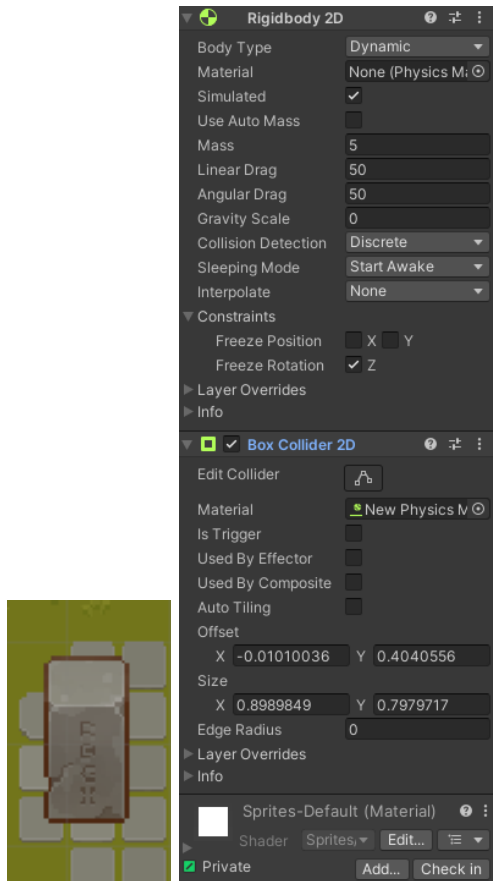
Enemy Logic and Behavior

- Enemies are controlled by the EnemyScript and EnemyProjectile scripts. This includes taking damage from the player, inflicting damage to the player, dropping loot, as well as any SFX associated with it.

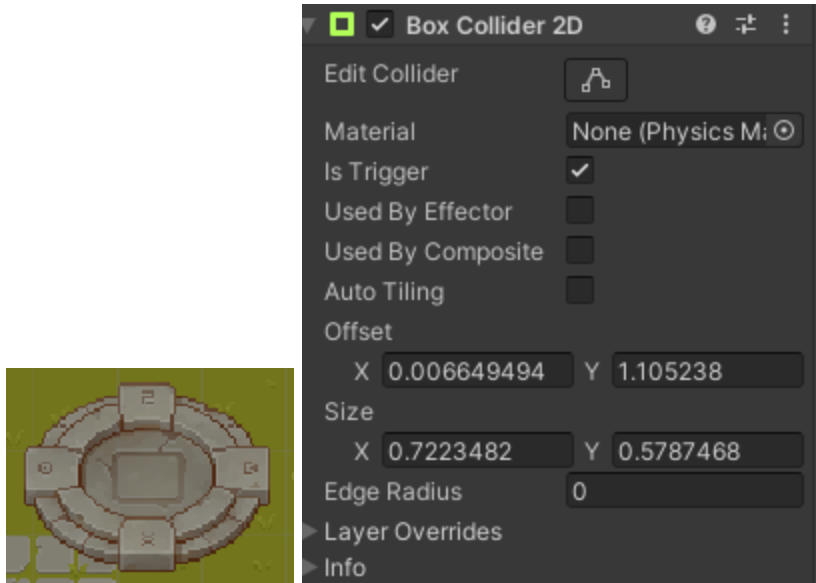


Puzzle Mechanic

- Everstrife had a simple puzzle setup involving three components: Key, Lock, and a Door.
- Key - the key uses a combination of a frictionless material, a tag, and colliders to work.



- Lock - the lock mainly uses a trigger collider. OnTriggerEnter2D detects if the key object enters the trigger, whereas OnTriggerExit2D checks if it's moved out of the trigger. The sprite renderer also changes texture, depending on the door state.



```
Unity Message | 0 references
private void OnTriggerEnter2D(Collider2D other)
{
    // opens door if key is on the pressure plate and change visual
    if (other.CompareTag("Key"))
    {
        Debug.Log("Open Sesame");
        doorCollider.isTrigger = true;

        spriteRenderer.sprite = openedDoor;
        keyRenderer.sprite = glowingKey;

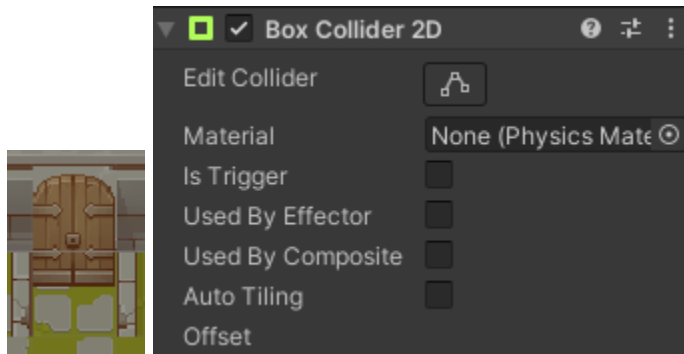
        playSFX();
    }
}

Unity Message | 0 references
private void OnTriggerExit2D(Collider2D other)
{
    // closes door if key is on the pressure plate and change visual
    if (other.CompareTag("Key"))
    {
        Debug.Log("Closed");
        doorCollider.isTrigger = false;

        spriteRenderer.sprite = closedDoor;
        keyRenderer.sprite = defaultKey;

        playSFX();
    }
}
```

- Door - by default, the door has an active collider that prevents the player from passing through, unless the puzzle is completed.



5 Technical Challenges

A decent chunk of our time went into resolving GitHub issues. Initially, we would have conflict issues that would cause it to overwrite our files. Gitignore was also not working properly, as it kept pushing files that it shouldn't be pushing, including UserSettings. We technically didn't resolve the issues and had to migrate to the native Unity Version Control, and everything seemed to work fine after that. Other challenges we faced include:

- Layering and depth simulation: Creating the game layers in a visual 2D environment required a lot of planning and testing
- AI behavior attuning: finding the right amount of enemy speed and distance of pathfinding to make a realistic game with realistic combat was a constant back and forth.
- Time constraints: as the project has essentially a 2 month development time, there was so many features we wanted to implement or planned to implement but had to either get a bare skeleton of it or just move onto something else entirely.

These challenges provided valuable experience for debugging, learning unity and developing our C# code skills.

6 Game Assets

6.1 Texture and Asset

Assets used include

1. "Pixel Art Potion pack - Animated" by Karsior
2. "Fantasy Wooden GUI: Free" by Black Hammer
3. "Pixel Art Top Down - Basic | Cainos" by Cainos.
4. Sprites made by us and AI generation.

Pixel art potion pack was used to make in-game potion items, as well as the inventory potion UI. Fantasy wooden GUI was used for our inventory tabs and pages as well as for button decoration in our game menu. Finally the Pixel Art Top Down was used for the general game texture which makes the layers of our game maps. Our sprites were either made by us or we used AI to generate them and then modified them using free open source graphic editing software like Krita.

6.2 SFX

All sound effects used in the game were all taken from the website called Pixabay. Credits to:

1. DRAGON-STUDIO
2. Dennish18
3. Djartmusic
4. Freesound_Community
5. RoyaltyFreeMusicStudio
6. SoundReality

7 Conclusion

Overall, Everstrife was a fun project to develop and successfully demonstrates how to make that classic Action adventure RPG experience we as gamers know and love. Throughout the development process, we learned concepts and features such as the input system, collider and physics objects, some AI pathfinding techniques, and so much more. Although due to time constraints, we were unable to flesh out the game further by adding more enemies, various biomes/levels, and, most importantly, implementing the faction system, which would be one of the game's core features.

If we were to redo this project, I think the only thing we would've done differently is use Unity's native Version Control instead of GitHub, since a lot of our time was spent resolving issues we couldn't solve until a few weeks into development.

8 Future Work

If given more time, we would work on fleshing out Pandamonia more by adding enemy variety, more biomes, NPCs with their corresponding factions, and more playable characters.

1. More enemies - we would add flying enemies, enemies that can use magic, or bosses with mechanics.
2. Expanding the world of Pandamonia with different regions.
3. Various biomes - we initially planned to add several biomes like bogs, abandoned towns, and mausoleums.
4. Factions - one of our intended features for the game was an allegiance system where the player could choose which factions to help, but this could result in the opposing faction turning hostile toward the player.
5. Player classes - we wanted to add 3 different classes to the game: ranger, necromancer, and mage knight. Due to time constraints, we were only able to add the ranger.

9 Appendix

Reflections

Zhire: Developing Everstrife was an enjoyable experience! This project taught me a lot, including, but not limited to, player input and movement, UI and SFX, colliders and physics, and a little bit of AI Pathfinding. I was also working on another Unity project alongside this one, so it even helped me with some parts of that project. There were some hiccups, specifically at the start of the semester, when our progress was hindered by GitHub issues, but it all worked out in the end. The only thing I would redo in this project is using Unity Version Control instead of GitHub. Overall, I really liked working on this project. Chris was a good teammate and a pleasure to work with.

Chris: Developing Everstrife on Unity was a blast and an excellent learning opportunity. Me and Zhire also worked together incredibly well and the smaller team size allowed us to stay on task and focused on what we needed to do. There were various tutorial by Dr. Sam Qorbani to get us started and help us wrap our minds around unity's physics and understand how to make a 2D game in a 3D format. I learned a lot about texture layers, item instantiation, prefabs, rigidbodies, hitboxes, texture and sprite editing, and just so much more of how C# works as a code language. We did have our snags of course, with Github being a horrible experience for merging our works together, however after getting some help and guidance from classmates we were able to export things into prefabs and reuse a lot of our work, we then switched to unity version control to keep this issue from happening in the future. It was an incredibly fun learning experience, I loved our game and am very proud of what we made, and Zhire was an outstanding team mate who I would love to work with again.